

A HANDS-ON, CODE-FIRST COURSE

LangChain From Zero

Build real GenAI apps with Gemini & local models

PHASE 8 — BUILDING A RAG APPLICATION

Prerequisites: Phases 4, 6, 7 done — LCEL composition, chunking, and a working FAISS store/retriever.

Targets: the RAG query flow · grounding prompt · format_docs · the LCEL RAG chain · returning sources

Estimated time for this phase: 80–120 minutes

Where this phase sits

Everything converges here. You have a retriever (Phase 7) and you know LCEL composition (Phase 4). This phase pipes them together so a model answers real questions **using your documents** — the single most common production use of LangChain.

Ph 0–4 ✓ **Setup** → **Fundamentals** → **Models** → **Prompts** → **Chains**

Ph 5 ✓ **Memory & Conversational Apps**

Ph 6 ✓ **Loading & Splitting Documents**

Ph 7 ✓ **Embeddings & Vector Stores** — FAISS store + retriever.

Ph 8 **Building a RAG Application** ← you are here
The query-time flow, grounding prompt, the full LCEL RAG chain, returning sources.

Ph 9 **Tools & Agents**

Ph 10–12 **LangGraph · Deployment · Capstone**

Phase 8 · Building a RAG Application

WHAT YOU'LL BE ABLE TO DO AFTER THIS PHASE

Describe the query-time RAG flow; write a grounding prompt that keeps the model honest; format retrieved chunks into context; assemble a complete RAG chain with LCEL on both Gemini and Ollama; and return the **sources** alongside each answer so users can verify it.

In this phase

1. 8.1 — The whole RAG picture, now at query time
2. 8.2 — Step 1: the grounding prompt
3. 8.3 — Step 2: formatting retrieved chunks
4. 8.4 — Step 3: the full RAG chain (LCEL)
5. 8.5 — Running it on Gemini and Ollama
6. 8.6 — Returning sources with the answer
7. 8.7 — Keeping answers grounded (reducing hallucination)
8. 8.8 — Common errors & troubleshooting
9. 8.9 — Mini practice exercise
10. Terms introduced this phase

8.1 — The whole RAG picture, now at query time

Phases 6–7 built the *ingestion* side (load → split → embed → store), done once. RAG's other half runs on *every question*:

QUERY TIME (every question)



In words: take the question, retrieve the most relevant chunks, paste them into a prompt as "context," and ask the model to answer using only that context. That's RAG. Notice the question goes to *two* places — the retriever *and* the prompt — which is exactly what Phase 4's `RunnableParallel` / `RunnablePassthrough` were built for.

8.2 — Step 1: the grounding prompt

The prompt is what makes RAG trustworthy. It has two blanks — `{context}` (the retrieved chunks) and `{question}` — and explicit instructions to answer *only* from the context.

```
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_template(
    """Answer the question using ONLY the context below.
    If the answer is not in the context, say "I don't know."

    Context:
    {context}

    Question: {question}"""
)
```

TERM: GROUNDING

Grounding means tying the model's answer to provided source text rather than its own memory. The instruction "use ONLY the context" and "say I don't know if it's not there" is what stops the model from inventing facts. This single prompt design choice is the difference between a reliable RAG bot and a confident liar.

8.3 — Step 2: formatting retrieved chunks

The retriever returns a *list of Document objects*, but the prompt's `{context}` blank needs a *string*. A tiny helper joins them:

```
def format_docs(docs) -> str:
    return "\n\n".join(doc.page_content for doc in docs)  # stitch chunk texts together
```

It pulls `.page_content` from each retrieved Document and joins them with blank lines. In the chain we'll place it right after the retriever so chunks become a clean context block.

8.4 — Step 3: the full RAG chain (LCEL)

Here's the complete, self-contained program — loading the FAISS store you saved in Phase 7, then building the RAG chain with LCEL.

```
from dotenv import load_dotenv
from langchain_community.vectorstores import FAISS
from langchain_google_genai import ChatGoogleGenerativeAI, GoogleGenerativeAIEmbeddings
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough

load_dotenv()
```

```

# 1. load the store + make a retriever (Phase 7)
embeddings = GoogleGenerativeAIEmbeddings(model="gemini-embedding-001")
store = FAISS.load_local("faiss_index", embeddings, allow_dangerous_deserialization=True)
retriever = store.as_retriever(search_kwargs={"k": 3})

# 2. the grounding prompt (8.2)
prompt = ChatPromptTemplate.from_template(
    """Answer the question using ONLY the context below.
    If the answer is not in the context, say "I don't know."

    Context:
    {context}

    Question: {question}"""
)

model = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=0) # 3 temp 0 =
factual

def format_docs(docs):
    return "\n\n".join(d.page_content for d in docs)

# 4. THE RAG CHAIN
rag_chain = (
    {"context": retriever | format_docs, "question": RunnablePassthrough()} # 5
    | prompt
    | model
    | StrOutputParser()
)

print(rag_chain.invoke("How do I get a refund?")) # 6 ask in plain English

```

Line/ Block	Explanation
1	Reload the saved store and expose it as a retriever returning the top 3 chunks. (You could also build it fresh here with the Phase 7 code.)
3	<code>temperature=0</code> : for factual Q&A you want consistent, grounded answers, not creativity.
5	The key line. A dict (auto- <code>RunnableParallel</code> , Phase 4) feeds the prompt's two blanks: <code>context</code> = run the retriever on the input <i>and</i> format the docs to a string; <code>question</code> = pass the original input through unchanged. Both get the same input — the question string.
4/6	The dict's output (<code>{context, question}</code>) flows into <code>prompt → model → parser</code> . <code>invoke</code> with a plain question string runs the whole pipeline and returns a grounded answer.

Expected output (depends on your document):

According to the context, refunds are processed within 30 days of your request.

WHY THE PARALLEL DICT?

The question must reach two destinations: the **retriever** (to find chunks) and the **prompt** (so the model sees the actual question). `{"context": retriever | format_docs, "question": RunnablePassthrough()}` sends the same input down both paths at once and reassembles them into the dict the prompt needs. This is the canonical RAG wiring — memorise its shape.

8.5 — Running it on Gemini and Ollama

The usual swap — but remember the **golden rule from Phase 7**: the embedding model must match the one your store was built with. So swap *both* the chat model and the embeddings (and load the matching Ollama-built index):

```
from langchain_ollama import ChatOllama, OllamaEmbeddings

embeddings = OllamaEmbeddings(model="nomic-embed-text") # matches the
                                                         # Ollama-built store
store = FAISS.load_local("faiss_index_ollama", embeddings,
allow_dangerous_deserialization=True)
retriever = store.as_retriever(search_kwargs={"k": 3})

model = ChatOllama(model="llama3.2", temperature=0) # local chat
model

# the prompt, format_docs, and rag_chain definition are IDENTICAL
```

FULLY LOCAL RAG

With Ollama for both embeddings and chat, your entire RAG app runs offline, free, and private — no data leaves your machine. That's a genuinely useful capability for sensitive documents, and you built it with the same chain you used for Gemini.

8.6 — Returning sources with the answer

A grounded answer is good; a grounded answer *with its sources* is production-ready — users can click through and verify. The trick: keep the retrieved documents around instead of discarding them. Use `RunnableParallel` + `.assign` (Phase 4) so the chain outputs both the answer and the source docs.

```
from langchain_core.runnables import RunnableParallel, RunnablePassthrough

# run retriever + keep the question; THEN compute the answer from both
rag_with_sources = RunnableParallel(
    context=retriever, # keep the Document objects (not
formatted)
    question=RunnablePassthrough(),
).assign( # add an "answer" key, keeping
context+question
    answer=(
        (lambda x: {"context": format_docs(x["context"]), "question": x["question"]})
```

```

    | prompt | model | StrOutputParser()
)
)

result = rag_with_sources.invoke("How do I get a refund?")

print(result["answer"])          # the grounded answer text
print("--- sources ---")
for doc in result["context"]:    # the chunks it used
    print(doc.metadata)         # e.g. {'source': 'handbook.pdf', 'page':
4}

```

Piece	Explanation
<code>RunnableParallel(context=retriever, question=...)</code>	Produces a dict holding the raw retrieved <code>Document</code> s and the question — both preserved.
<code>.assign(answer=...)</code>	Adds an <code>answer</code> key without dropping <code>context</code> / <code>question</code> . Inside, a lambda formats the docs to text, then the familiar <code>prompt model parser</code> produces the answer.
Result	<code>{"context": [Documents], "question": "...", "answer": "..."} </code> — answer plus the exact chunks (and their metadata) behind it.

THIS IS THE PHASE 4 PAYOFF

Everything here — parallel branches, passthrough, assign, a lambda to reshape — came from Phase 4. RAG-with-sources is just those primitives applied to a real problem. If the wiring feels dense, revisit §4.2–4.4; the shapes are identical.

8.7 — Keeping answers grounded (reducing hallucination)

TERM: HALLUCINATION

A **hallucination** is when a model states something false with confidence. RAG reduces it by giving the model real source text — but doesn't eliminate it. These levers help:

Lever	Why it helps
Strong grounding prompt (§8.2)	Explicit "use ONLY the context / say I don't know" curbs invention.
<code>temperature=0</code>	Less random $\Rightarrow$ less likely to drift from the source.
Return sources (§8.6)	Lets users (and you) catch wrong answers fast.
Good chunking (Phase 6) & right <code>k</code>	If the relevant text isn't retrieved, the model can't use it. Tune chunk size and <code>k</code> .

Better retrieval	If answers miss, inspect what the retriever returned <i>first</i> — most RAG bugs are retrieval bugs, not model bugs.
------------------	---

DEBUGGING RAG: LOOK AT RETRIEVAL FIRST

When a RAG answer is wrong, the instinct is to blame the model. Usually the retriever simply didn't surface the right chunk. Before touching the prompt, print `retriever.invoke(question)` and check whether the answer is even *in* there. If not, fix chunking/embeddings/ `k` — not the model.

8.8 — Common errors & troubleshooting

Symptom	Likely cause & fix
Model answers "I don't know" for things clearly in your docs	Retrieval missed the chunk. Print <code>retriever.invoke(q)</code> ; increase <code>k</code> , adjust chunk size/overlap (Phase 6), or check the store was built from the right docs.
Model ignores context and answers from memory	Weak prompt. Strengthen the grounding instruction (§8.2) and set <code>temperature=0</code> .
<code>KeyError: 'context'</code> or <code>'question'</code>	The dict keys feeding the prompt must exactly match the template blanks. Check spelling in both places.
Garbage / irrelevant retrieved chunks	Embedding model mismatch between build and query (Phase 7 golden rule), or the question is off-topic for your docs.
Context too long / token-limit error	Too many or too-large chunks. Lower <code>k</code> or <code>chunk_size</code> .
<code>format_docs</code> errors with "str has no page_content"	You passed already-formatted text where Documents were expected. In the sources chain, keep <code>context=retriever</code> (raw Docs) and format inside the lambda.

8.9 — Mini practice exercise

YOUR TURN (DO THIS BEFORE PHASE 9)

Task: Build `rag_app.py` over a document you care about (your notes, a manual, a paper). Load your Phase 7 store, build the RAG chain, and wrap it in a `while True` question loop (like Phase 5's chatbot, but each question is independent). Print the answer *and* the source metadata for every question. Ask it: (a) a question whose answer is in the doc, (b) a question phrased with completely different words than the doc uses, and (c) a question the doc *doesn't* cover — and confirm it says "I don't know" instead of inventing.

Why: this is a genuinely useful, complete application — a Q&A bot over your own data with citations. It's also the core of the Phase 12 capstone, so getting it solid now pays off twice.

Hints (not the full answer):

- Use the `rag_with_sources` version (§8.6) so you have both answer and sources to print.

- Test (c) deliberately — a good "I don't know" is a feature, not a failure. If it invents instead, strengthen the prompt.
- If (b) fails, that's a retrieval issue: inspect `retriever.invoke(question)` and tune `k` / chunking.
- Try swapping Gemini ↔ Ollama (with the matching store) to feel the speed/quality trade-off on your own data.

Terms introduced this phase

Term	One-line meaning
RAG chain	The query-time pipeline: retrieve chunks → stuff into prompt → model → answer.
Grounding	Tying answers to provided source text rather than the model's memory.
Grounding prompt	A prompt instructing the model to answer only from <code>{context}</code> .
Context	The retrieved chunk text inserted into the prompt for the model to use.
format_docs	Helper joining retrieved Documents' <code>page_content</code> into one string.
RunnablePassthrough (in RAG)	Sends the question to the prompt unchanged while the retriever also gets it.
Return-sources pattern	<code>RunnableParallel</code> + <code>.assign</code> to output answer <i>and</i> the chunks used.
Hallucination	A confident but false model statement; RAG reduces but doesn't remove it.
k (top-k)	How many chunks the retriever returns per query.
Retrieval-first debugging	When answers are wrong, inspect what was retrieved before blaming the model.

PHASE 8 COMPLETE

You built a complete RAG application — grounding prompt, formatted context, the canonical LCEL RAG chain, and a sources-returning variant — running on both Gemini and a fully-local Ollama stack. You also know that most RAG bugs are retrieval bugs. **Next — Phase 9: Tools & Agents**, where the model stops just answering and starts *acting*: you'll give it custom tools, let it decide when to call them, and build a tool-using agent with the modern `create_agent` API (the replacement for the legacy `AgentExecutor`).